NAME

grouper - find groups of identical or similar directories in a sumtag database

SYNOPSIS

grouper --database *DB* [*ACTIONS*] [*OPTIONS*]

DESCRIPTION

grouper finds groups of directories whose contents are identical or nearly so -- for example, two slightly different versions of the same project saved at slightly different times. It is an experimental companion to **sumtag**(1): it works entirely from a sumtag SQLite database (see the **--database** option of **sumtag**(1)), never reading file contents or extended attributes and never computing a checksum itself. The database must have been populated by sumtag first.

Alongside sumtag's own tables, grouper owns a set of derived tables inside the same database file (**dirs**, **dir_files**, **dir_pairs**, **groups**, **group_dirs**, **grouper_meta**). All of them are rebuildable from sumtag's tables, and all can be dropped with **--clean-db**.

The work is a pipeline of three explicit stages, each persisted so later stages are cheap to re-run:

1. **--index** distills the file rows into the distinct directories that directly contain at least one stamped file. Only direct children count -- a directory's own file listing is its signature.

2. **--pairs** compares every directory to every other with one named comparison function (see **COMPARISON FUNCTIONS** below) and stores every nonzero similarity. This is the expensive stage, so the similarity threshold is deliberately not applied here: regrouping at a different threshold recomputes nothing.

3. **--threshold** walks the stored pairs best-first and partitions the directories into groups -- one placement decision per directory, and groups are never merged. At equal similarity, the pair sharing more files ranks first, so large overlaps seed groups ahead of trivial ones. The result is persisted and reported.

--prep runs stages 1 and 2 together. A bare **grouper --database** *DB* with no action reports the stored grouping again. Stages may be combined in one invocation (**--prep --threshold** 0.7).

OPTIONS

--database *DB*

Path to the sumtag SQLite database. Required on every invocation.

--prep

Stages 1+2: **--index** then **--pairs** in one shot.

--index
Stage 1: (re)build the directory index. Silent on success.

--pairs
Stage 2: compare every directory to every other and store all nonzero similarities. Silent on success. Scoring fans out across **--jobs** worker processes; every database write stays in the parent. The stored pair set is identical at any **--jobs** value.

--threshold *X*
Stage 3: build and persist the grouping from the stored pairs, using *X* (0.0-1.0) as the similar-enough bar, then report it. Skipped when the stored grouping is already current for this threshold and comparison function -- in that case it just reports. A threshold below a stored **--min-sim** floor is refused with a hint to re-run **--pairs**, never answered silently from an incomplete pair table.

--fn *FN*
Comparison function for **--pairs**/**--compare** (see **COMPARISON FUNCTIONS** below). Default: **name-score**.

--jobs *N*
Worker processes for **--pairs** scoring (default: all CPUs); 1 disables multiprocessing. Each worker loads its own signature table from the database, so every worker holds a full copy in memory -- on a large corpus this option is a memory knob as much as a speed knob. Small corpora run serially regardless.

--max-df *N*
For **--pairs**: instead of exhaustive all-pairs comparison, score only candidate pairs that share at least one signature token (a basename or digest, depending on the comparison function) present in at most *N* directories. Candidates get exact scores from full signatures -- the cap governs who gets looked at, never what a look sees; what is given up is only pairs whose sole overlap is ubiquitous tokens, whose similarity is necessarily tiny. By default, comparison is exhaustive up to roughly ten thousand directories, beyond which candidate mode engages with *N*=1000; 0 forces exhaustive at any size. Candidate mode is announced on standard error and the cap is recorded in the database.

--min-sim *X*
For **--pairs**: keep only pairs scoring at least *X* (0.0-1.0). By default every nonzero pair is stored so that **--threshold** stays a free exploratory knob, but at archive scale the weak tail dwarfs everything interesting. The floor governs what is kept, never what is looked at; scores are still computed exactly. A later **--threshold** below the floor is refused (see above).

--name

For **--pairs**/**--compare**: two directories whose basenames differ score 0, unconditionally -- only same-named directories can pair, and therefore group. The typical prey is versioned copies of one tree (**proj**/, **backup/proj**/, **old/proj**/), where the directory's own name survives a copy even as contents drift. Also sharply reduces what **--pairs** stores.

--progress

Live progress bar on standard error for the build stages (**--index**/**--pairs**), following **sumtag**(1)'s **--progress** conventions: appears once a stage has run two seconds, redrawn in place, cleared on completion, suppressed when standard error is not a terminal. Shows done/total with rate, percent, elapsed time, and an ETA.

--ls *DIR*

List the files in *DIR* (absolute or mount-relative) using the directory index.

--compare *DIR_A DIR_B*

Print the similarity (0.0-1.0) of two directories' contents.

--dups

Report duplicate files (same digest).

--top [*N*]

Report the *N* (default 1) most frequently occurring checksums and their files, excluding empty files. Mutually exclusive with **--dups**.

--min *N*

For **--dups**/**--top**: only report groups of at least *N* files (default: 2).

--clean-db

Drop every grouper-owned derived table and **VACUUM**, returning their space -- **dir_pairs** in particular can dwarf the sumtag tables it was derived from. All-or-nothing on purpose: the artifacts are interdependent, so a partial clean would leave a grouping that cannot be reported. Sumtag's own tables are never touched. Cannot be combined with any other action.

COMPARISON FUNCTIONS

Similarity is size-normalised: two identical 10,000-file archives and two identical 3-file folders both score 1.0. Directories with no signature overlap always score 0.

digest

Content only; a renamed file still matches.

name-digest
All-or-nothing per (name, digest) pair: a file counts only when both its basename and its digest agree.

name-score
The default. Name-anchored partial credit: one point for a shared basename, two more if the digests also agree.

INTERRUPTION

An interrupted **--pairs** run keeps its completed inserts, but the run is not marked finished: a partial pair table can never masquerade as a complete one, and **--threshold** refuses it until a **--pairs** run completes.

EXIT STATUS

0 on success; **1** when an error prevents the request from being answered (no stored pairs, a threshold below the stored floor, an unknown directory, and so on). Errors are reported on standard error.

EXAMPLES

Build everything and group at 0.7:

grouper --database foo.sqlite --prep --threshold 0.7

Regroup at a looser threshold (recomputes nothing):

grouper --database foo.sqlite --threshold 0.5

Report the stored grouping again:

grouper --database foo.sqlite

Compare two directories:

grouper --database foo.sqlite --compare /data/proj /backup/proj

Prepare a very large corpus, keeping only meaningful pairs between same-named directories:

grouper --database big.sqlite --prep --name --min-sim 0.4 --progress

Return the space the derived tables occupy:

grouper --database foo.sqlite --clean-db

NOTES

grouper is experimental. Its derived tables go stale when sumtag rescans the filesystem; re-run the pipeline (**--prep**) to refresh them. Duplicate detection is scoped to a single digest algorithm at a time: two identical files stamped under different algorithms are not found as duplicates.

SEE ALSO

sumtag(1), **dedupe**(1)

AUTHOR

The sumtag authors.